# Reviewer Pack — Minimal Tropical Growth Rate of Polynomial Systems vs AC0 Pa…

Entry 6bb39aa47f7f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 18:37:32 UTC

## Minimal Tropical Growth Rate of Polynomial Systems vs AC0 Parity Circuit Size

**Entry ID**: `6bb39aa47f7f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 18:37:32 UTC

### 1. Conjecture

**Field A** (mathematical branch): Tropical Geometry × Algebraic Combinatorics **Field B** (complexity object): Complexity Theory: AC0 Circuit Complexity

**Statement**:

> {'one': 'Let P be a polynomial system over the tropical semiring, where each polynomial has degree d in its variables.', 'two': 'Define the minimal tropical growth rate of P as g(P) = min_{p∈P} log_2(max(p)).', 'three': 'For any AC0 parity circuit C with n inputs and size s, there exists a polynomial system P such that g(P) ≥ c·log(s) for some constant c.'}

**Rationale (proposer's reasoning)**:

> {'one': 'Tropical geometry provides a framework to study the growth of functions on the real line.', 'two': 'Algebraic combinatorics can encode properties of AC0 circuits as polynomial systems.', 'three': 'By linking these fields, we may find an invariant that captures the complexity of AC0 parity circuits.'}

**Taxonomy category**: `AC0_PARITY` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `e9c30d39aa289abb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if, for all generated AC0 parity circuits C with n inputs and size s ≤ 40, g(P) ≥ c·log(s) holds true for at least 80% of seeds, where g(P) is the minimal tropical growth rate of the associated polynomial system P. The conjecture is falsified if any seed produces a g(P) < c·log(s) or if less than 70% of seeds meet this condition.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | UNCERTAIN | 1.00 | UNCERTAIN | HITS |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - `'tropical geometry' AND 'algebraic combinatorics' AND 'AC0 circuit complexity' AND 'polynomial systems'` - `'minimal tropical growth rate' AND 'AC0 parity circuit size' AND 'tropical semiring' AND 'polynomial degree'` - `'complexity theory' AND 'AC0 circuit' AND 'polynomial system' AND 'minimal tropical growth rate'`

**Top relevant hits considered**: - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/nlin/0101006v1] On Complexity and Emergence - [http://arxiv.org/abs/2406.12990v2] Universal Early-Time Growth in Quantum Circuit Complexity

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_ac0_circuit(n, s):
    if n == 1:
        return [random.choice([0, 1])]
    elif s == 1:
        return [random.choice([0, 1])]

    left = generate_ac0_circuit(n // 2, s // 2)
    right = generate_ac0_circuit(n // 2, s - s // 2)

    if len(left) != len(right):
        raise ValueError("Left and right circuits must have the same length")

    return [left[i] | right[i] for i in range(len(left))]

def compute_polynomial_system(circuit):
    n = len(circuit)
    m = 1 << n
    P = []

    for i in range(m):
```

```
        term = 0
        for j in range(n):
            if (i >> j) & 1:
                term |= circuit[j]
        P.append(term)

    return P

def min_tropical_growth_rate(P):
    max_p = max(P)
    if max_p == 0:
        return 0
    return math.log2(max_p)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []

    for n in [5, 10, 15, 20, 30, 40]:
        for s in range(1, min(n + 1, 41)):
            try:
                circuit = generate_ac0_circuit(n, s)
                P = compute_polynomial_system(circuit)
                g_P = min_tropical_growth_rate(P)
                results.append((n, s, g_P))
            except Exception as e:
                return {
                    "metric_name": "minimal_tropical_growth_rate",
                    "metric_value": None,
                    "instances_tested": 0,
                    "conjecture_holds": False,
                    "counterexample": str(e)
                }

    if not results:
        return {
            "metric_name": "minimal_tropical_growth_rate",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid circuits generated"
        }

    total_g_P = sum(g_P for _, _, g_P in results)
    instances_tested = len(results)

    mean_g_P = total_g_P / instances_tested
    conjecture_holds = all(g_P >= math.log2(s) for _, s, g_P in results)

    return {
        "metric_name": "minimal_tropical_growth_rate",
        "metric_value": mean_g_P,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not all(results):
        return

    mean_g_P = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_g_P} std=NA support_fraction={support_fraction}")
    elif support_fraction < 0.7:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='g(P) < c·log(s)' first_failing_seed={first_faili
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2268961e.py", line 106
    return
    ^^^^^^
SyntaxError: 'return' outside function
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: NONE

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 11298 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 6181 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 5183 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 5674 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 14223 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 9508 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 9376 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 10403 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 8995 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 80841 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/6bb39aa47f7f.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6bb39aa47f7f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/6bb39aa47f7f.tar.gz` (if generated)